

7 查找 (检索)

7.1 查找的基本概念

1. 定义

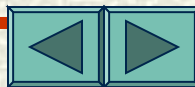
确定给定值 k 在含有 n 个记录的文件中的位置的过程称为查找。

2. 查找算法的度量

用平均查找长度 ASL 来度量：
$$ASL = \sum_{i=1}^n P_i C_i$$

其中， n ：记录个数， P_i ：查找第 i 个记录的查找概率， C_i ：找到第 i 个记录的比较次数。

若 $P_i=1/n$ （等概），则
$$ASL = \frac{1}{n} \sum_{i=1}^n C_i$$



7.7 查找

7.2 线性查找（顺序查找）

1. 方法

从第一个记录开始，逐个比较记录值：

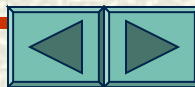
找到与 k 相等的记录，查找成功；

否则，查找失败。

例：（5，13，17，47，55，70，94）

查找 $k=55$ ，比较 5 次，查找成功，

查找 17，则失败。



7 查找

7.2 线性查找

1. 算法描述

```
SeqSearch(r[], n, k)
{
    r[n].key=k;    i=0;
    while (r[i].key<>k) i++;
    return(i);
}
```

说明：若返回 n 表明查找不成功。

2. 性能分析

$$ASL = \sum_{i=1}^n P_i C_i \stackrel{\text{等概}}{=} \frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2}$$



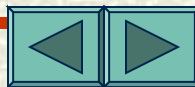
7 查找

7.3 对分查找

——又称二分查找、折半查找，属于有序表的查找。

1. 基本思想

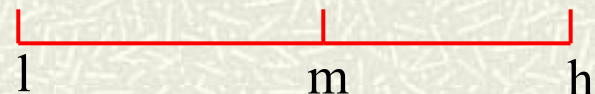
先找到“中间记录”，比较其关键字，如果关键字与给定值 K 相等，则查找成功；否则中间结点将线性表分成两个子表，若关键字小于 K ，则在前半部分子表中查找；反之在后半部分子表中查找。这样把搜索区间缩小了一半。重复以上过程，直到找到与 k 相等的关键值。（递增排列情况）



7 查找

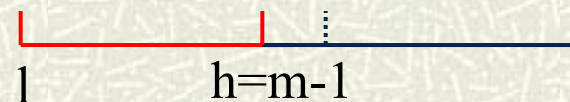
7.2 对分查找

1. 方法



1) 初始化: 设顺序表 $r[1:n]$, low 一搜索范围下界, $high$ 一搜索范围上界, mid 一中间位置 $(low+high)/7$

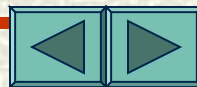
7) $r[mid].key=k$: 查找成功;



$r[mid].key>k$: 令 $high=mid-1$ 继续查找;

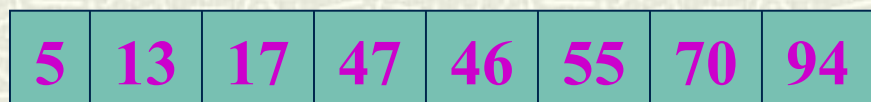
$r[mid].key<k$: 令 $low=mid+1$ 继续查找。

3) 直到 $low>high$, 查找失败



例:

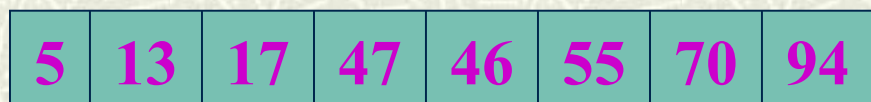
k=55



low

mid

high



low mid

high

查找成功

k=17



low

mid

high



low mid high



low mid high



high low

查找失败

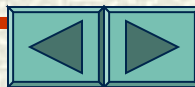


7 查找

7.3 对分查找

1. 算法描述

```
BinSearch(r[], n, k)
{
    low<-0; hig<-n-1;
    while (low<hig) {
        mid=(low+hig)/2;
        if (k==r[mid].key)
            return(1);
        if (k<r[mid].key) hig=mid-1;
        if (k>r[mid].key) low=mid+1;
    }
    return(0);
}
```

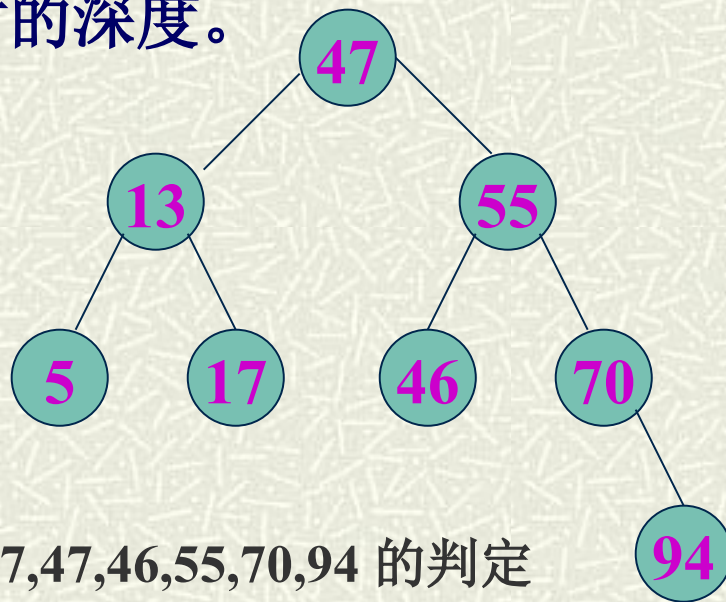


7 查找

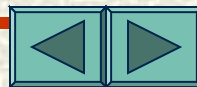
7.3 对分查找

2. 算法分析

♣ 对分查找的判定树：把比较次数相同的记录放在同一层次，各层次之间用分支相连，可得到一棵二叉树，称为**判定树**。对分查找恰好是一条从判定树根到被查结点的一条路径，而比较次数恰好是树的深度。



序列： 5,13,17,47,46,55,70,94 的判定
树



7 查找

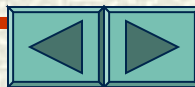
对分查找

3. 算法分析

♣ 等概率下的平均查找长度:

$$ASL = \sum P_i C_i \stackrel{\text{等概}}{=} \frac{1}{n} \sum C_i = \frac{1}{n} \sum k \cdot 2^{k-1} = \frac{n+1}{n} \log_2(n+1) - 1$$
$$\approx \log_2(n+1) - 1, \quad (n > 50)$$

n<30 时，对分查找的优势不明显



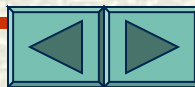
7 查找

7.4 分块查找（索引顺序查找）

- ♣ 将数据分成若干块
- ♣ 块与块间数据有序
- ♣ 块内数据无序

1. 算法思想

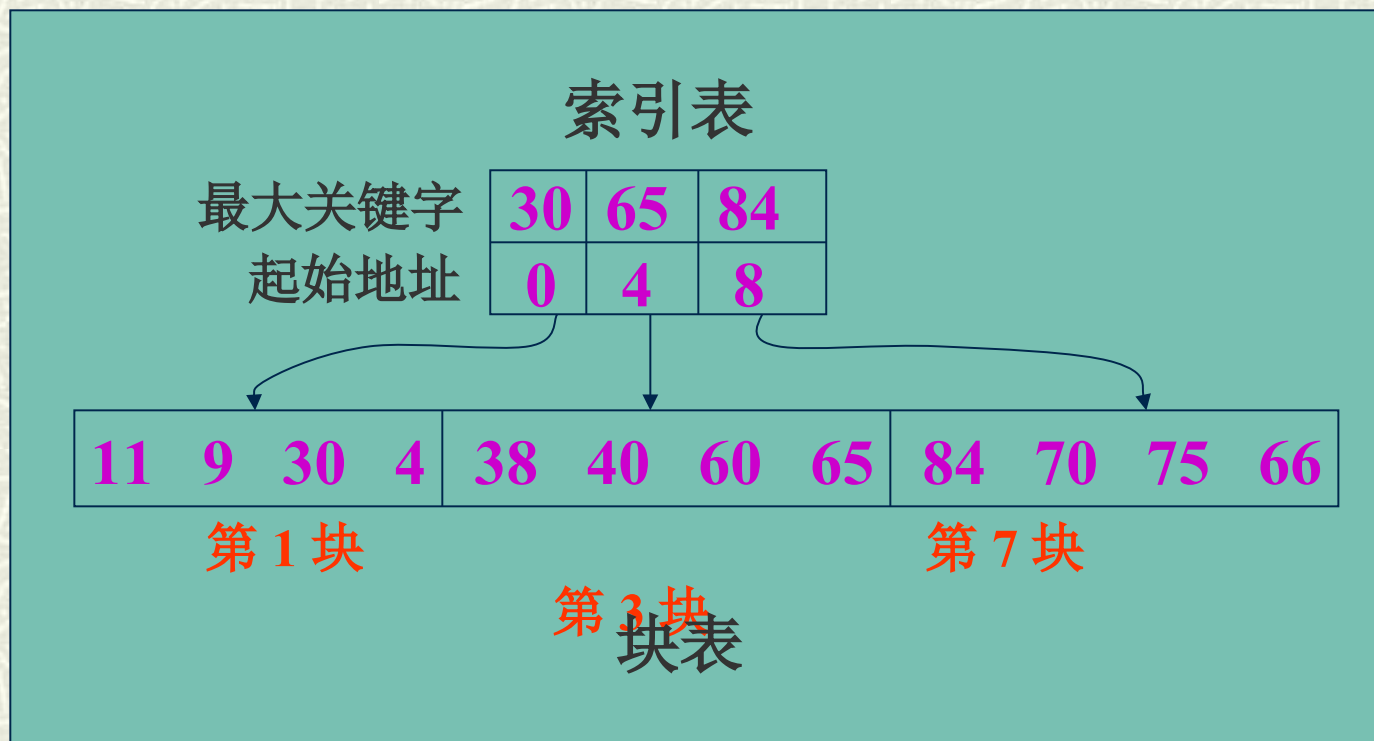
- ① 将各块中的最大关键字构成一个递增有序的索引表
- ② 先查找索引表（采用对分或顺序查找），以确定所在块
- ③ 在所在块中进行顺序查找



7 查找

7.4 分块查找

例：



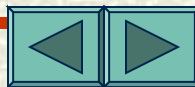
7 查找

7.4 分块查找（索引顺序查找）

3. 算法分析

$$\begin{array}{ccccc} \text{平均查找长度} & & \text{索引表平均查找长度} & & \text{块表平均查找长度} \\ \uparrow & & \uparrow & & \uparrow \\ ASL & = & ASL_b & + & ASL_n \\ & & & & \\ & & & & \approx \log_2(b+1) - 1 + \frac{s+1}{2} \approx \log_2\left(\frac{n}{s} + 1\right) + \frac{s}{2} \end{array}$$

其中， $r[1:n]$ 分为 b 块，每块中的记录数 $s=n/b$



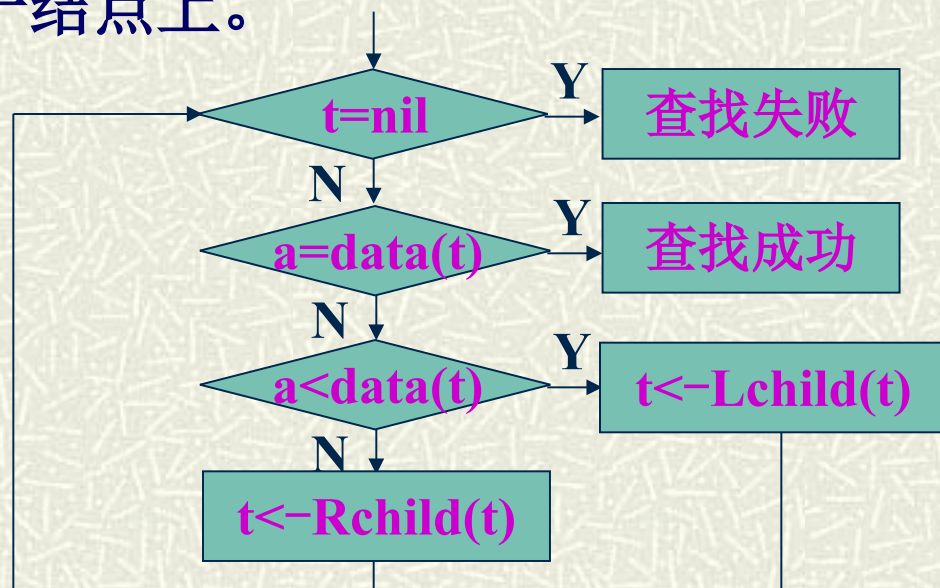
7 查找

7.5 二叉排序树查找

1. 基本思想（动态查找）

和对分查找类似，二叉排序树可看成是一个有序结构，查找过程是走一条从根结点到所在结点的路径。若查找不成功，则可将被查找元素插入到二叉排序树的叶子结点上。

2. 查找方法



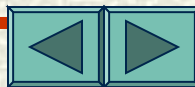
7 查找

7.5 二叉排序树查找

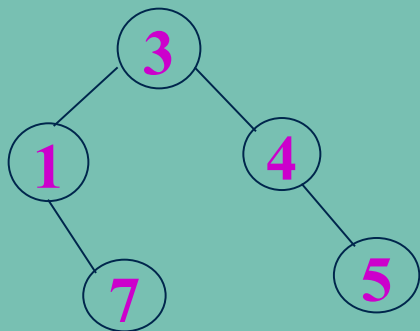
2. 算法描述

```
BstSearch(t, k)
{
    p=t;
    while (p) {
        if (k==p->data) return(1);
        if (k<p->data) { q=p; p=p-
>lchild; }
        if (k>p->data) { q=p; p=p-
>rchild; }
    }
    s=malloc(sizeof(nodetype));
    s->data=k; s->lchild=s->rchild=NULL;
    if (t==NULL) p=s;
    else if (k<q->data) q->lchild=s;
        else q->rchild=s;

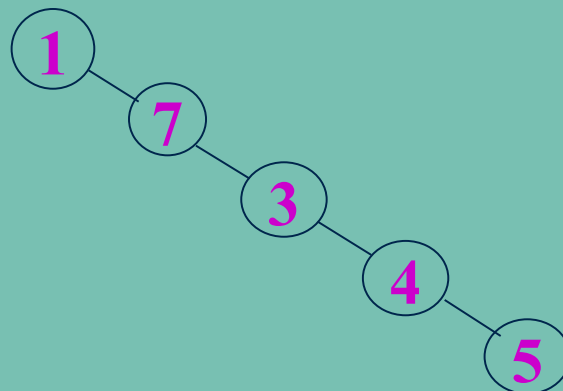
    return(0);
}
```



7.5



$$ASL = (1 + 7 \times 7 + 3 \times 7) / 5 = 7.7$$



$$ASL = (1 + 7 + 3 + 4 + 5) / 5 = 3$$

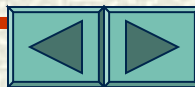
3. 算法分析

♣ 与各结点的比较次数 = 路径长度 + 1

平均查找长度:
$$ASL = \frac{1}{n} \left[\sum_{i=1}^n (\text{路径} + 1) \right]$$

♣ 由于相同序列构成的二叉排序树不唯一，故平均查找长度亦不同（极端情况为线性表）。

ASL 最小的二叉排序树形态为除最下层以外，其余各层都是充满的。



7 查找

7.6 哈希表技术及其查找

1. 哈希表（散列表）

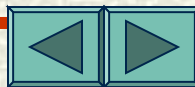
1) **基本思想**：不需查找，通过对给定值作某种运算，直接求得关键字等于给定值的记录的位置。

2) 概念

哈希 (Hash) 函数：关键字 key 与存储位置间的对应关系函数，记为 $H(key)$ 。

哈希地址：即 $H(key)$ ，或称散列地址。

哈希表：若用一维数组来存放文件，则 $H(key)$ 为数组的下标，该一维数组称为哈希表。



7 查找

7.6 哈希表技术及其查找

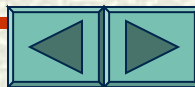
3) 装填系数

$$\alpha = n/m$$

n — 记录个数, m — 哈希表长
一般取: $\alpha=0.65 \sim 0.85$ 。

4) 哈希查找要解决的问题

- ♣ 哈希函数的构造
- ♣ 解决冲突



7 查找

7.6 哈希表技术及其查找

2. 哈希函数的构造

1) 数字分析法: 关键字已知, 分析数字的分布, 选择数字分布均匀的若干位作为哈希地址 (适合静态数据)。

8 1 3 4 6 5 3 7

8 1 3 7 7 7 4 7

8 1 3 8 7 4 7 7

8 1 3 0 1 3 6 7

8 1 3 7 7 8 1 7

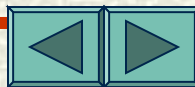
8 1 3 3 8 9 6 7

8 1 3 5 4 1 5 7

分析: 第 1, 7, 3 位分布太不均匀, 删去; 第 8 位只有 7 和 7, 删去。第 4, 5, 6, 7 位分布近似随机, 设存储空间为 100, 则可取其中二位作为哈希地址:

65, 77, 74, 13, 78, 89,

41



7 查找

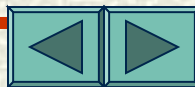
7.6 哈希表技术及其查找

7) 平方取中法：取关键字平方后的中间几位作为哈希地址（一个数平方后的中间几位数和数的每一位都相关）。

关键字： 0100, 1100, 1700, 1160, 7061, 7067

关键字²： 0010000, 1710000, 1440000, 1345600, 4747771, 4751844

哈希地址： 010, 710, 440, 345, 747, 751



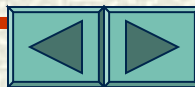
7 查找

7.6 哈希表技术及其查找

3) 除留余数法: $H(\text{key}) = \text{key} \% p$

其中 p 为 $\leq m$ 的质数。

注意: m 为哈希表长



7 查找

7.6 哈希表技术及其查找

4) 折叠法：将关键字分为等长的几段（除最后一段外），再将各段相加作为哈希地址。

- ♣ 移位折叠：将各段向左边靠齐后相加。
- ♣ 边界折叠：将奇数字段或偶数字段倒排后相加。

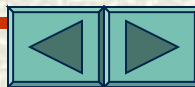
关键字值：17370374111770，分为5段：173, 703, 741, 117, 70

移位折叠：

173
703
741
117
70
—
879

边界折叠：

173
307
741
711
70
—
897



7 查找

7.6 哈希表技术及其查找

3. 解决冲突的方法

1) 线性探测再散列（开放地址法）

设 hash 表： $T[0:m-1]$ ， 哈希函数： $H(key)$

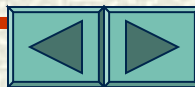
冲突时同义词中的另一个地址为：

$$d_{j+1} = (d_1 + j) \% m \quad (j=1, 2, \dots, s, s \geq 1)$$

其中 $d_1 = H(key)$

若完成一个循环仍未找到空间，则溢出。

说明：这种方法易造成关键字聚集，增加查找时间。



7 查找

7.6 哈希表技术及其查找

查找算法如下:

LineSearch(T, k, m)

{

$i = H(k)$; $j = i$;

 while ($T[j].key \neq k$ && $T[j].key \neq 0$) {

$j = (j+1) \% m$;

 if ($j == i$) { 表满; return(-1); }

 }

 return(j);

}

若 $T[j].key=0$ 表示此地址为空。



例： 设 hash 表装填系数 $a=0.75$ ， 则表长 $m=n/a=19$

一组记录： (13, 79, 01, 73, 44, 55, 70, 84, 77, 68, 11, 10, 79, 14)

Hash 地址:	0	1	7	3	4	5	6	7	8	9	10	11	17	13	14	15	16	17	18
Hash 表:	68	01		70	55		73				44	77	79	13	11	10	84	79	14
比较次数:	1	1		1	1		1				1	7	1	1	4	6	1	7	5

$K=01$, $addr=01\%17=01$, 填入

$addr=10+1=11$, 冲突

$addr=10+7=17$, 冲突

$addr=10+3=13$, 冲突

$addr=10+4=14$, 冲突

$addr=10+5=15$, 冲突

$addr=11+6=17$, 填入

$$ASL = (1 \times 9 + 7 + 4 + 6 + 7 + 5) / 14 = 7.36$$



7 查找

7.6 哈希表技术及其查找

7) 平方探测再散列（二次探测再散列）

冲突时求地址公式为：

$$d_{7j} = (d_1 + j^2) \% m$$

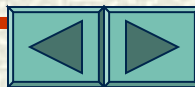
$$(j=1, 2, \dots, s, s \geq 1)$$

$$d_{7j+1} = (d_1 - j^2) \% m$$

3) 随机探测再散列

本方法是用一组预先给定的随机数求冲突时求地址，公式为： $d_j = (d_1 + R_j) \% m$ ， $(j=1, 2, \dots, s, s \geq 1)$

其中 R_j 为一组随机数列

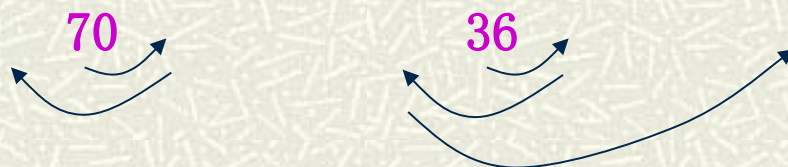


例： 设一组关键字 {37, 75, 63, 48, 94, 75, 36, 18, 70, 54} ， 其中 $a=0.67$ ， 采用 hash 函数： $H(key)=key \% 13$ ， 采用平方探测再散列解决冲突， 构造 hash 表， 并计算其

ASL 。

解： $n=10, a=0.6$ ， 则 $m=n/a=15$

Hash 地址：	0	1	7	3	4	5	6	7	8	9	10	11	17	13	14
Hash 表：			54	94	70	18	37			48	75	63	75		36
比较次数：					1	1	3	1	1				1	1	1
4															



$$ASL = (1 \times 8 + 3 + 4) / 10 = 1.5$$



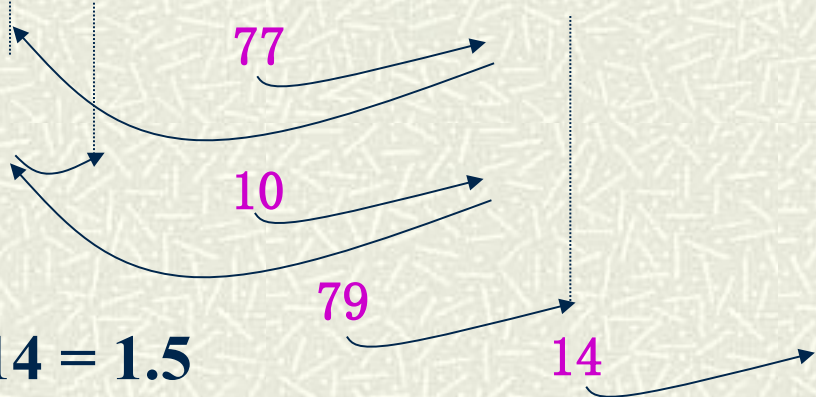
例：一组关键字：

(13, 79, 01, 73, 44, 55, 70, 84, 77, 68, 11, 10, 79, 14)

$n=14$ ，选取 $a=0.75$ ，则哈希表长 $m=19$ ，用除留余数法构造哈希函数，选 $p=17$ ，用随机探测法解决冲突，其中随机数序列为：3, 16, 55, 44, ...

解：

Hash 地址:	0	1	7	3	4	5	6	7	8	9	10	11	17	13	14	15	16	17	18
Hash 表:	68	01		70	55		73	77	10		44	1	79	13	79		84	14	
比较次数:		1	1		1	1		1	3	4		1	1	1	1	7		1	7



$$ASL = (1 \times 10 + 3 + 4 + 7 + 7) / 14 = 1.5$$



7 查找

7.6 哈希表技术及其查找

4) 链地址法

哈希表为 $T[0..m-1]$ ，设置一个由 m 个指针分量组成的一维数组 $ST[0..m-1]$ ，凡哈希地址为 i 的记录都插入到头指针为 $ST[i]$ 的链表中。

链地址法是个动态结构，更适合在造表之前无法确定记录个数的情况。 参见图 7.77



7 查找

7.6 哈希表技术及其查找

查找算法如下：

```
ChnSearch(ST, m, k)
{
    i=H(k);  p=ST[i];
    while (p && p->data!=k) p=p->next;
    return(p);
}
```

返回值 p 指向查找元素的地址，若 p=NULL，说明表中无此元素，可将其插入到哈希表中。



7 查找

7.6 哈希表技术及其查找

4. 性能分析

- ♣ 采用不同的解决冲突方法，平均查找长度也可能不同。
- ♣ 与线性查找、对分查找相比，ASL 较小。
- ♣ 平均查找长度是装填系数 a 的函数。适当选择 a 的值是很关键的。

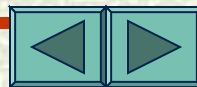
线性探测：

$$ASL = \frac{1}{2} \left(1 + \frac{1}{1-a} \right)$$

随机或平方探测：

$$ASL = -\frac{1}{a} \ln(1-a)$$

与 a 有关， a 越小，冲突越少，但浪费空间越大



7 查找

7.7 小结

- ♣ 理解有关概念
- ♣ 掌握线性查找算法
- ♣ 掌握对分查找算法
- ♣ 了解分块查找
- ♣ 掌握二叉排序树查找
- ♣ 理解哈希表有关概念和哈希函数的构造
- ♣ 掌握 hash 表的构造方法和解决冲突的方法
- ♣ 理解哈希表的查找性能

